

Analysis of Algorithms

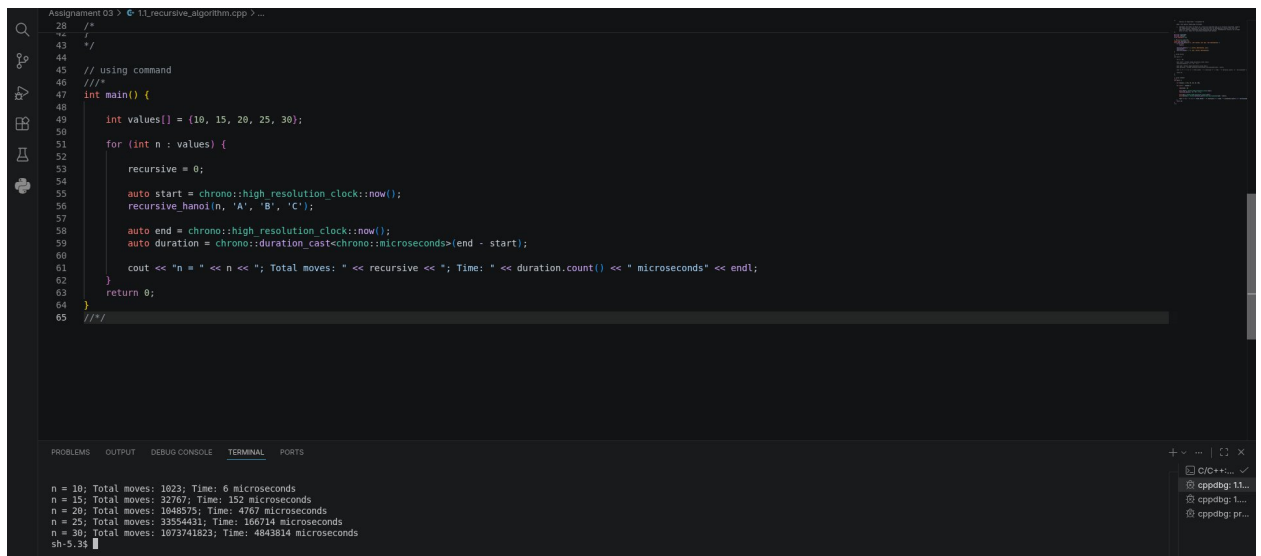
Assignment 03

Name: Jose Ignacio Cabascango Fernandez

Link GitHub: <https://github.com/CodeWithJose1/Analysis-of-Algorithms/tree/main/Assignment-03/src>

1. Implement the Towers of Hanoi as a recursive algorithm and as an iterative algorithm. Compare the computational complexity of the algorithm with the mathematical analysis and with the empirical analysis techniques discussed during the lecture. Mathematical analysis was already done in class. Check for consistency between both methods.

1.1 Recursive algorithm

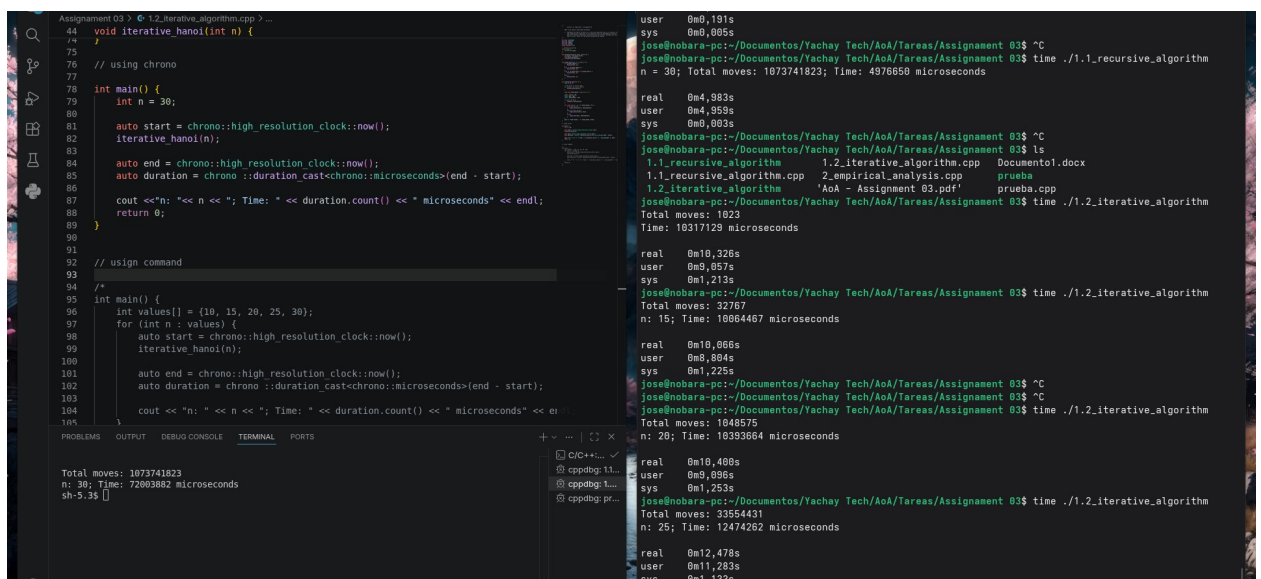


```
28 /*
29  *
30  */
31 // using chrono
32 // using command
33 // using chrono
34 int main() {
35     int values[] = {10, 15, 20, 25, 30};
36     for (int n : values) {
37         recursive = 0;
38         auto start = chrono::high_resolution_clock::now();
39         recursive_hanoi(n, 'A', 'B', 'C');
40         auto end = chrono::high_resolution_clock::now();
41         auto duration = chrono::duration_cast<chrono::microseconds>(end - start);
42         cout << "n = " << n << ", Total moves: " << recursive << ", Time: " << duration.count() << " microseconds" << endl;
43     }
44     return 0;
45 }
46 //
```

n = 10; Total moves: 1023; Time: 6 microseconds
n = 15; Total moves: 32767; Time: 152 microseconds
n = 20; Total moves: 1048575; Time: 4967 microseconds
n = 25; Total moves: 33554431; Time: 166714 microseconds
n = 30; Total moves: 1073741823; Time: 4843814 microseconds
sh-5.35

Image 1: Recursive algorithm and data visualization with chrono.

1.2 Iterative algorithm



```
44 void iterative_hanoi(int n) {
45     // using chrono
46     // using command
47     // using chrono
48     int main() {
49         int n = 30;
50         auto start = chrono::high_resolution_clock::now();
51         iterative_hanoi(n);
52         auto end = chrono::high_resolution_clock::now();
53         auto duration = chrono::duration_cast<chrono::microseconds>(end - start);
54         cout << "n = " << n << ", Time: " << duration.count() << " microseconds" << endl;
55         return 0;
56     }
57 // using command
58 // using chrono
59 // using command
60 // using chrono
61 int main() {
62     int values[] = {10, 15, 20, 25, 30};
63     for (int n : values) {
64         auto start = chrono::high_resolution_clock::now();
65         iterative_hanoi(n);
66         auto end = chrono::high_resolution_clock::now();
67         auto duration = chrono::duration_cast<chrono::microseconds>(end - start);
68         cout << "n = " << n << ", Time: " << duration.count() << " microseconds" << endl;
69     }
70     return 0;
71 }
72 // using command
73 // using chrono
74 // using command
75 // using chrono
```

Total moves: 1073741823
n: 30; Time: 72093882 microseconds
sh-5.35

user 0m0.191s
sys 0m0.005s
jose@nobar-pc:~/Documentos/Yachay Tech/AoA/Tareas/Assignment 03\$ time ./1.1_recursive_algorithm
n = 30; Total moves: 1073741823; Time: 4576560 microseconds
real 0m4.983s
user 0m4.959s
sys 0m0.003s
jose@nobar-pc:~/Documentos/Yachay Tech/AoA/Tareas/Assignment 03\$ ls
1.1_recursive_algorithm 1.2_iterative_algorithm.cpp Document1.docx
1.1_recursive_algorithm.cpp 2_empirical_analysis.cpp prueba
1.2_iterative_algorithm 'AoA - Assignment 03.pdf' prueba.cpp
jose@nobar-pc:~/Documentos/Yachay Tech/AoA/Tareas/Assignment 03\$ time ./1.2_iterative_algorithm
Total moves: 1023
Time: 10317129 microseconds
real 0m10.328s
user 0m9.057s
sys 0m1.213s
jose@nobar-pc:~/Documentos/Yachay Tech/AoA/Tareas/Assignment 03\$ time ./1.2_iterative_algorithm
Total moves: 32767
n: 15; Time: 10864467 microseconds
real 0m10.066s
user 0m9.804s
sys 0m1.225s
jose@nobar-pc:~/Documentos/Yachay Tech/AoA/Tareas/Assignment 03\$ time ./1.2_iterative_algorithm
Total moves: 1048575
n: 20; Time: 18393664 microseconds
real 0m10.408s
user 0m9.066s
sys 0m1.253s
jose@nobar-pc:~/Documentos/Yachay Tech/AoA/Tareas/Assignment 03\$ time ./1.2_iterative_algorithm
Total moves: 33554431
n: 25; Time: 12474262 microseconds
real 0m12.478s
user 0m11.283s
sys 0m1.133s

Image 2: Iterative algorithm and data visualization with command.

Compare the computational complexity:

Mathematically, the Towers of Hanoi requires exactly:

$$M(n) = 2^n - 1$$

moves. Therefore, both the recursive and iterative implementations have the same asymptotic computational complexity:

$$\Theta(2^n)$$

The experimental results confirm this because both algorithms generated exactly the theoretical number of moves for every tested value of n .

n	Real Moves	Recursive Moves	Iterative Moves	Recursive Time (μs)	Iterative Time (μs)
10	1023	1023	1023	6	10182903
15	32767	32767	32767	152	10504918
20	1048575	1048575	1048575	4767	10401415
25	33554431	33554431	33554431	166714	12165585
30	1073741823	1073741823	1073741823	4843814	71747437

Table 1: System Time Comparison using chrono.

This exact correspondence verifies the mathematical analysis.

Recursive implementation

The recursive algorithm shows a clear exponential increase in execution time $n = (10, 15, 20, 25, 30)$. The same tendency is observed with the ‘time’ command (see the Table 2). The recursive execution increases from approximately **0.009 s** for $n = 10$ to **4.983 s** for $n = 30$.

This behavior is consistent with:

$$T(n) = \Theta(2^n)$$

For example, increasing n by 5 multiplies the theoretical number of operations by approximately:

$$2^5 = 32$$

which explains the rapid growth in execution time.

Iterative implementation

The iterative implementation also performs exactly $2^n - 1$ moves, so its theoretical complexity is:

$$\Theta(2^n)$$

However, its measured execution time is considerably higher than the recursive implementation. For example, using chrono see the Table 1, recursive and iterative timer.

The iterative algorithm is much slower in this particular implementation. This does not mean that its asymptotic complexity is worse. Both algorithms still perform the same exponential number of Hanoi moves.

The large difference is most likely caused by the implementation itself. Your iterative version uses structures such as vector, accesses to the last element, pop_back(), push_back(), and repeated legal-move checks. These operations introduce additional overhead for every movement.

Comparison using the time command

The system measurements show the same general behavior:

n	Recursive				Iterative			
	Time (µs)	Real	User	Sys	Time (µs)	Real	User	Sys
10	36	0m0.009s	0m0.003s	0m0.006s	10317129	0m10.326s	0m9.057s	0m1.213s
15	1088	0m0.008s	0m0.004s	0m0.004s	10064467	0m10.066s	0m8.804s	0m1.225s
20	31470	0m0.041s	0m0.033s	0m0.007s	10393664	0m10.400s	0m9.096s	0m1.253s
25	190358	0m0.197s	0m0.191s	0m0.005s	12474262	0m12.478s	0m11.283s	0m1.133s
30	4976650	0m4.983s	0m4.959s	0m0.003s	73496640	1m13.506s	1m10.880s	0m2.222s

Table 2: System Time Comparison using command.

Recursive

For $n = 30$:

- real = 4.983 s
- user = 4.959 s
- sys = 0.003 s

The ‘user’ time is very close to the ‘real’ time, which indicates that most of the execution was spent running the algorithm itself rather than waiting for system operations.

Iterative

For $n = 30$:

- real = 73.506 s
- user = 70.880 s
- sys = 2.222 s

The iterative implementation takes considerably longer.

However, remember that real, user, and sys measure the complete program, while your chrono measurement can measure specifically the Hanoi section. Therefore, their values do not need to be identical.

Consistency between mathematical and empirical analysis

The mathematical and empirical analyses are consistent in the most important aspect: both algorithms require an exponential number of moves.

Mathematical analysis:

$$T(n) = 2^n - 1 = \Theta(2^n)$$

Empirical analysis:

$n \Rightarrow$ execution time increases exponentially

Furthermore, both programs generated exactly the expected number of moves:

$$2^n - 1$$

for every tested value of n .

Therefore, the experimental results support the mathematical prediction that both implementations have:

$$\boxed{\text{Time Complexity} = \Theta(2^n)}$$

In conclusion, the mathematical and empirical analysis is consistent. Both the recursive and iterative implementations generate exactly $2^n - 1$ moves, and therefore have an exponential complexity of $\Theta(2^n)$. The recursive implementation was faster in the experiments, mainly because the iterative implementation uses `std::vector` to perform `pop_back()` to remove the disk from one tower and `push_back()` to insert it into another, verifying the movements. This difference affects execution time, but does not change the asymptotic computational complexity.

2. Apply empirical analysis to the “linear congruential method” for creating pseudo-random numbers. Research about the algorithm, the textbook gives some information to start up with.

The linear congruential method (LCM) generates pseudo-random numbers, this is because these numbers are calculated using a recurrence formula which can be calculated mathematically:

$$x_{i+1} = (ax_i + c) \bmod m$$

This recurrence uses the following parameters to generate pseudo-random numbers:

- seed: This is the initial value of the sequence, that is, x_0 . The first pseudorandom number is calculated from this value.
- a (multiplier): This is used to multiply the previous value x_i before adding the increment. Its choice influences the sequence and its period.
- c (increment): This is added after multiplying x_i by a .
- m (modulo): The $\%m$ operation limits the generated values to the range: $0 \leq x_i < m$.

For the experiment, the following parameters were used are:

- Seed: 37
- Multiplier a : 45
- Increment c : 93
- Modulus m : 87

The number of generated values n was changed, and the LCG parameters were left fixed. The execution time was measured using the `chrono` library, following the empirical analysis approach presented in the lecture.

n	Last value	Execution time (μs)
1000	69	15
10000	24	158
100000	42	1,535
1000000	18	20,227
10000000	12	125,418
100000000	33	1,029,274

Table 3: Experimental Results

Empirical Analysis

The experimental results obtained show that the execution time increases as n is increased. For example, when n is increased from 1,000 to 10,000, the time also increases from 15 μs to 158 μs , which represents approximately a factor of 10.53. Similarly, increasing n from 10,000 to 100,000 increases the execution time from 158 μs to 1,535 μs , approximately a factor of 9.71.

Although the execution time does not increase by exactly the same factor as n in every experiment, the general trend is approximately linear. Variations in the measurements are expected in real computer systems because execution time can be affected by system activity and other processes.

Complexity Analysis

The main operation of the algorithm is executed once for each generated value:

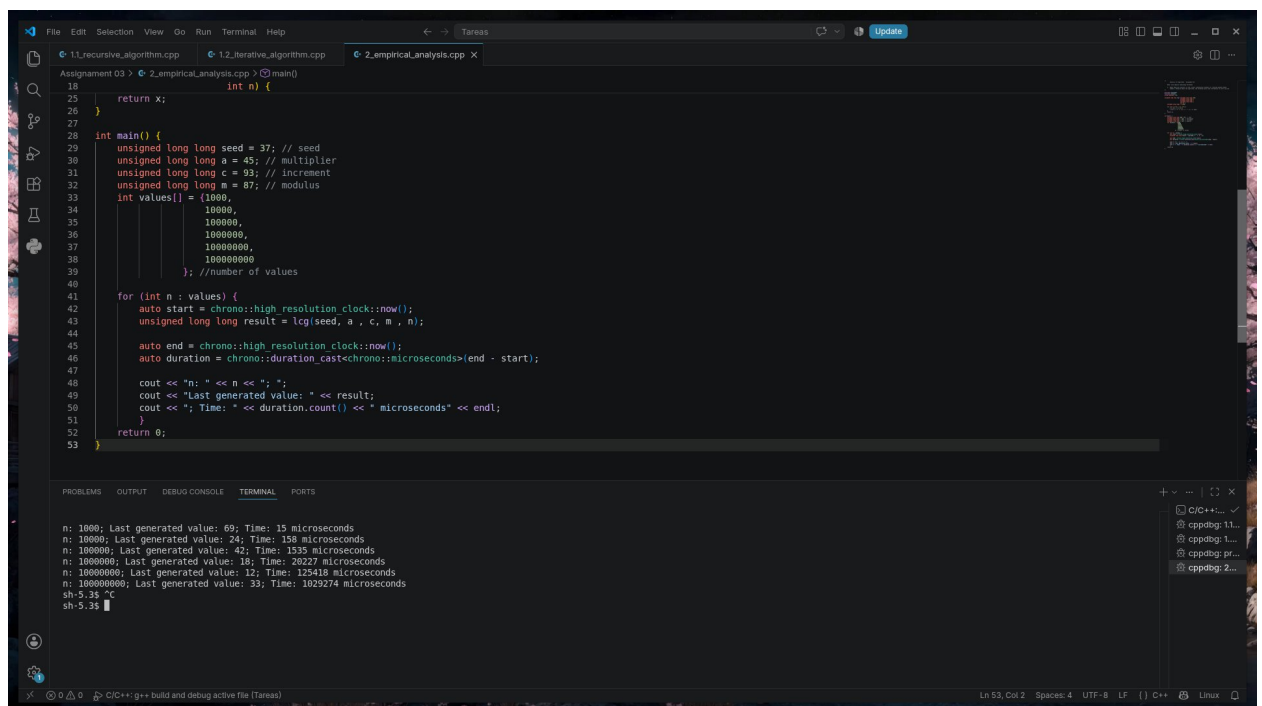
$$x = (ax + c) \bmod m$$

The for loop executes n times, and each iteration performs a constant amount of work. Therefore, the mathematical time complexity is:

$$T(n) = \Theta(n)$$

The empirical results are consistent with this theoretical analysis because the execution time generally increases approximately proportionally to the number of generated values.

The empirical results are consistent with this theoretical analysis because the execution time generally increases approximately proportionally to the number of generated values. Finally, the empirical analysis supports the mathematical analysis of the Linear Congruential Method. Generating (n) pseudo-random values requires a linear number of iterations, resulting in a time complexity of $(\Theta(n))$. The experimental measurements confirmed the expected linear growth, although the measured times are not perfectly proportional due to variations in the execution environment.



The screenshot shows a C++ IDE with a file named `2_empirical_analysis.cpp`. The code implements a Linear Congruential Method (LCG) to generate pseudo-random values. It defines a function `lcg` that takes a seed, multiplier `a`, increment `c`, and modulus `m` as input, and returns a sequence of values. The `main` function sets up the LCG with `seed = 37`, `a = 45`, `c = 93`, and `m = 87`. It then iterates over a list of values `n` (1000, 10000, 100000, 1000000, 10000000) and for each `n`, it generates `n` values using the LCG. The execution time for each iteration is measured using `chrono::high_resolution_clock` and printed to the terminal. The terminal output shows the following results:

```
n: 1000; Last generated value: 69; Time: 15 microseconds
n: 10000; Last generated value: 24; Time: 158 microseconds
n: 100000; Last generated value: 42; Time: 1535 microseconds
n: 1000000; Last generated value: 18; Time: 20227 microseconds
n: 10000000; Last generated value: 12; Time: 125418 microseconds
n: 100000000; Last generated value: 33; Time: 1023274 microseconds
```

Image 3: Linear congruential method.